arXiv:2211.12881v1 [cs.AI] 23 Nov 2022

DGEKT: A DUAL GRAPH ENSEMBLE LEARNING METHOD FOR KNOWLEDGE TRACING

ChaoranCui, YumoYao, ChunyunZhang, HeboMa
Shandong University of Finance and Economics
Jinan, China
crcui@sdufe.edu.cn

YulingMa
Shandong Jianzhu University
Jinan, China
mayuling@mail.sdu.edu.cn

ZhaochunRen
Shandong University
QingDao, China
zhaochun.ren@sdu.edu.cn

Chen Zhang
Hong Kong Polytechnic University
Hong Kong, China
jason-c.zhang@polyu.edu.hk

James Ko
Education University of Hong Kongn
Hong Kong, China
jamesko@eduhk.hk

ABSTRACT

Knowledge tracing aims to trace students' evolving knowledge states by predicting their future performance on concept-related exercises. Recently, some graph-based models have been developed to incorporate the relationships between exercises to improve knowledge tracing, but only a single type of relationship information is generally explored. In this paper, we present a novel Dual Graph Ensemble learning method for Knowledge Tracing (DGEKT), which establishes a dual graph structure of students' learning interactions to capture the heterogeneous exercise-concept associations and interaction transitions by hypergraph modeling and directed graph modeling, respectively. To ensemble the dual graph models, we introduce the technique of online knowledge distillation, due to the fact that although the knowledge tracing model is expected to predict students' responses to the exercises related to different concepts, it is optimized merely with respect to the prediction accuracy on a single exercise at each step. With online knowledge distillation, the dual graph models are adaptively combined to form a stronger teacher model, which in turn provides its predictions on all exercises as extra supervision for better modeling ability. In the experiments, we compare DGEKT against eight knowledge tracing baselines on three benchmark datasets, and the results demonstrate that DGEKT achieves state-of-the-art performance.

***K*eywords** knowledge tracing, dual graph structure, graph convolutional networks, online knowledge distillation

1 Introduction

According to the statistics of UNESCO[1], more than 1.5 billion students worldwide have been affected by school and university closures due to the COVID-19 pandemic. In response to this crisis, online learning, which serves as an alternative to traditional face-to-face learning, is developing on an unprecedented scale. It is increasingly accessible for students to study on all sorts of online learning platforms, including Coursera, Udacity, and edX.

In the meantime, it has become crucial for online learning platforms to automatically estimate students' *knowledge states*, i.e., their mastery level of different knowledge concepts (e.g., the concepts of linear equation, power function, and inequality in mathematics) [1]. This enables a personalized learning experience to be provided for each student, such as learning resource recommendation [2], adaptive testing [3], and student profiling [4].

Knowledge tracing is one of the major research directions to address this problem. Although the mission is to monitor students' evolving knowledge states during the learning process, it is difficult to dynamically obtain the true knowledge states of students [5]. As an alternative, knowledge tracing predicts students' future performance on concept-related

[1] https://en.unesco.org/covid19/educationresponse/globalcoalition

知识追踪流程图

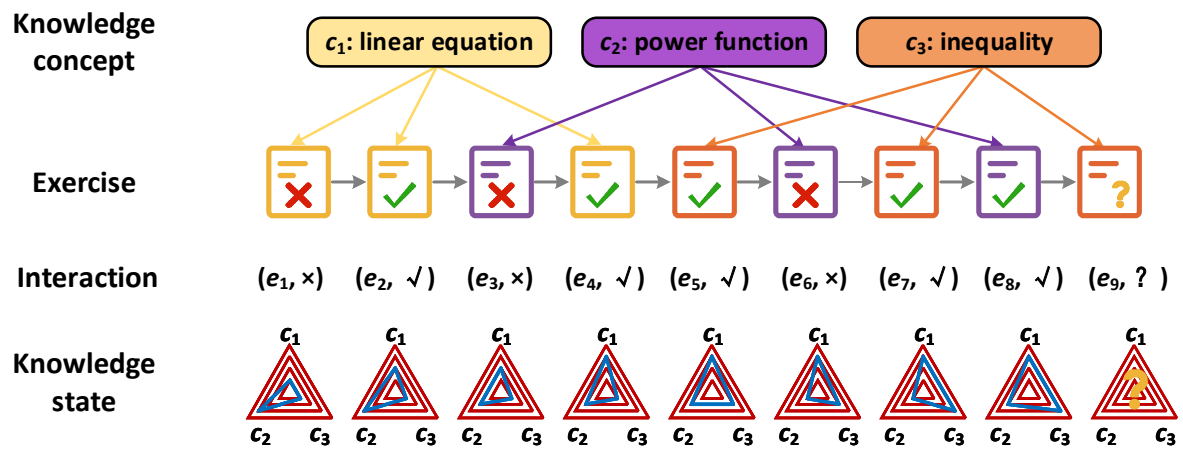


Figure 1: A simple schematic diagram of knowledge tracing. Based on a student's learning sequence of concept-related exercises, knowledge tracing predicts her/his responses to the following exercises and indirectly models the process by which the student's knowledge state of each concept changes, which is represented by a series of radar maps.

exercises based on their past interactions. In this way, students' knowledge states of each concept can be indirectly traced by observing their answers on the exercises related to the concept. As illustrated in Fig. 1, knowledge tracing models take the exercise-answering sequence of a student as the input. Generally, most models simplify the learning environments by assuming that the student's answers are only binary (i.e., correct or incorrect responses). Given a new exercise, the output of knowledge tracing models is the probability that the student correctly answers the exercise.

In the literature, knowledge tracing has been investigated for decades. At the early stage, some studies adopted the hidden Markov model to capture how students' knowledge state changes, among which the most popular method is bayesian knowledge tracing (BKT) [6, 7]. With the rise of deep learning, deep knowledge tracing (DKT) introduced recurrent neural networks (RNNs) to knowledge tracing and achieved promising results due to its ability to model long dependencies between learning interactions [8]. After that, more types of neural networks were used to improve knowledge tracing, such as memory-augmented neural networks [9, 10] and transformer networks [11, 12]. Recently, motivated by the observation that knowledge concepts are not isolated but interrelated to each other, the relationship information was incorporated as an inductive bias to improve knowledge tracing. Especially with the popularity of graph convolutional networks (GCNs) [13], different exercises and their dependencies are typically viewed as nodes and edges in a graph, and the influence between exercises is modeled via the node representation learning applied on the graph.

In spite of the encouraging progress, graph-based knowledge tracing methods remain unsatisfactory, and the technology is still in its infancy. A major limitation of existing research is the focus on a single type of relationship information, and the oversight of the fact that exercises are broadly connected via a variety of relationships. The potential benefits of integrating different relationship information for knowledge tracing are under-explored. In addition, only the exercise embeddings are generated through GCNs in existing research, which need to be further concatenated with the corresponding answer embeddings to represent students' learning interactions before they are fed into knowledge tracing models [14, 15]. As pointed out in [8], the separate embeddings of exercises and answers may be incompatible and degrade the performance.

In this paper, distinguished from prior studies relying on the dependencies between exercises, we investigate the relational graph of students' learning interactions, in which each node directly represents the combination of which exercise is answered and whether it is answered correctly or not. As a result, the intermediate step of separately acquiring and integrating the exercise and answer embeddings is skipped. More importantly, we establish a dual graph structure of learning interactions to capture their heterogeneous relationships. As shown in Fig. 2, a concept association hypergraph (CAHG) is constructed to reveal the associated knowledge concepts of learning interactions. Intuitively, a group of learning interactions can be made due to the mastery or non-mastery of the same knowledge concept. In the hypergraph [16], a hyperedge expresses a group-wise relationship linking multiple nodes simultaneously. Meanwhile, a directed transition graph (DTG) is used to describe the directed transitions from one learning interaction to another according to the learning sequences of all students. This is inspired by the observation that owing to the relatively short time interval between them, two consecutive interactions are more likely to be triggered in students' knowledge states with little change. In this way, the underlying collaborative effects across students are implicitly exploited. By propagating and aggregating the information over the dual graph structure, the hypergraph convolutional networks [17]

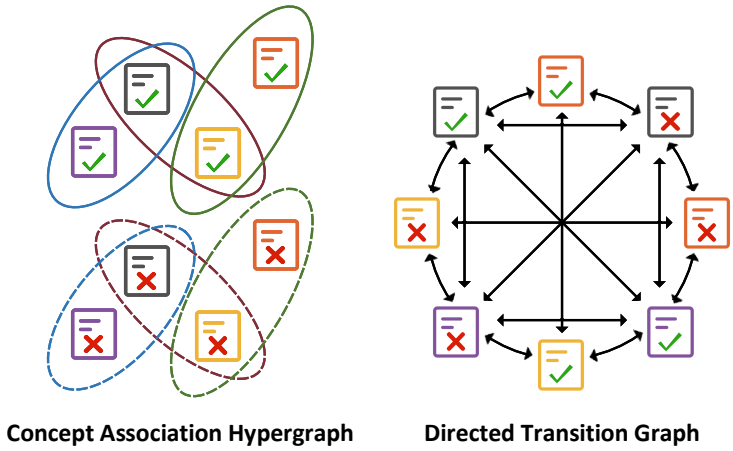


Figure 2: The dual graph structure of learning interactions, which consists of a concept association hypergraph (CAHG) revealing the associated knowledge concepts of learning interactions, as well as a directed transition graph (DTG) describing the directed transitions from one learning interaction to another in students' learning sequences.

and directed graph convolutional networks [18] are deployed to learn the node embeddings, respectively. Furthermore, the hidden representations of students' knowledge states can be expressed in CAHG and DTG from two independent perspectives.

For a comprehensive framework, it is logical to combine the dual graph models. The most straightforward way may be the concatenation- or adding-based fusion of their outputs. But in this study, we innovatively introduce the idea of online knowledge distillation [19] to achieve this end. Recall that knowledge tracing models aim to assess a student's knowledge state by predicting the student's performance on the exercises related to different concepts. However, due to the use of the student's unique historical learning sequence, knowledge tracing models can only be trained with respect to the prediction accuracy of the response to a single exercise at each step. With online knowledge distillation, we regard the dual graph models as peer student models and adaptively ensemble them to build a stronger teacher model using a gating mechanism [20]. Both student and teacher models are trained as usual by optimizing the prediction on the next exercise. Additionally, the teacher's predictions on *all exercises* are distilled back to student models as extra supervision, leading to more powerful modeling capacity. During training, the dual graph models can exchange their information and learn from each other via the ensemble teacher model; in testing, the stronger teacher model is deployed to achieve better prediction accuracy.

In a nutshell, we present a Dual Graph Ensemble learning method for Knowledge Tracing (DGEKT), which makes three main contributions:

- We propose a novel dual graph structure of students' learning interactions for knowledge tracing, based on which DGEKT captures the heterogeneous exercise-concept associations and interaction transitions via hypergraph modeling and directed graph modeling, respectively.

- We leverage online knowledge distillation to form an ensemble teacher model by adaptively combining the dual graph models, which in turn provides its predictions on all exercises as extra supervision for higher modeling capacity. To the best of our knowledge, DGEKT is the first effort to incorporate knowledge distillation techniques to knowledge tracing.

- We conduct extensive experiments on three benchmark datasets in comparison with eight previous methods. The experimental results demonstrate that DGEKT achieves state-of-the-art performance for knowledge tracing.

The remainder of the paper is structured as follows. Section 2 reviews the related work. Section 3 presents the proposed method for knowledge tracing. Experimental results and analysis are reported in Section 4, followed by the conclusion in Section 5.

2 Related Work

In this section, we first extensively review the existing literature on knowledge tracing. Then, we present a brief overview of two fields of graph convolutional networks and knowledge distillation, which are closely related to our work.

2.1 Knowledge Tracing

Due to its great potential to support intelligent educational services, knowledge tracing has emerged as a hot research topic in recent decades. In general, prior studies on knowledge tracing can be grouped into three categories: traditional non-deep learning methods, deep sequential modeling methods, and deep graph-based methods.

2.1.1 Traditional Methods

As the pioneering work for knowledge tracing, BKT [6] was proposed based on the hidden Markov model that used binary variables to represent students' knowledge states, i.e., whether a knowledge concept is mastered or not. BKT considered four factors affecting students' responses: the initial knowledge level, learning rate, guessing probability, and slipping probability. After that, many researchers extended BKT by introducing other important factors, such as students' individual variation in ability [7], the difficulty level of exercises [21], and prerequisite hierarchies and relationships within knowledge concepts [22]. Another typical kind of traditional methods is logistic models [23], which directly adopted logistic functions to predict the probability of correctly answering exercises according to the side information about students and knowledge concepts. Furthermore, knowledge tracing machines (KTM) [24] was developed to take advantage of factorization machines [25] to generalize logistic models to characterize the pairwise feature interactions. Despite the initial breakthrough of traditional methods, they mostly relied on linear models with restricted functional forms [8], which inevitably hindered the advancement of knowledge tracing.

2.1.2 Sequential Modeling Methods

With the huge surge of deep learning, different types of neural networks have been explored to model the learning sequences of students for knowledge tracing. DKT [8] was the first study that used the hidden state of RNNs at each step to represent students' evolving knowledge state and achieved much improved results over traditional methods. Unlike DKT modeling knowledge states via a single vector, dynamic key-value memory networks (DKVMN) [9] introduced external memory matrices to store the knowledge concepts and update the corresponding knowledge mastery of students. Due to the popularity of the attention mechanism [26] in deep learning, Pandey et al. [11] proposed a self-attentive model for knowledge tracing (SAKT), which applied the self-attention mechanism to identify the past learning interactions relevant to the exercise to be answered, and made the prediction by focusing on these relevant ones. Similarly, Ghosh et al. [12] presented a context-aware attentive knowledge tracing (AKT) model, which used the transformer network [27] to capture the long-term dependencies between students' learning interactions, and designed a monotonic attention mechanism with an exponential decay curve to reduce the importance of interactions in the distant past. Besides, some studies integrated deep sequence models with more elaborate factors related to learning, including the text content of exercises [28], temporal cross-effects between knowledge concepts [29], and students' individual cognition and knowledge acquisition sensitivity to exercises [30].

2.1.3 Graph-based Methods

It is well recognized that different exercises are not isolated but correlated to each other [5]. Therefore, some graph-based learning methods were developed to incorporate the relationship information to improve knowledge tracing. For example, Nakagawa et al. [14] presented the graph-based knowledge tracing (GKT), which established the graph between knowledge concepts in a statistics- or learning-based manner and modeled students' proficiency in knowledge concepts with GCNs by formulating the task as a time-series node-level classification problem. Deep hierarchical knowledge tracing (DHKT) [31] augmented DKT by separately learning exercise and concept embeddings with an auxiliary task of predicting the associations between exercises and knowledge concepts. As a further step, Liu et al. [15] proposed a pre-training model named PEBG that learned the pre-training embeddings by constructing the exercise-concept bipartite graph and simultaneously exploiting the exercise-concept relationships, inter-exercise similarities, and inter-concept similarities. In addition, Tong et al. [32] proposed a structure-based knowledge tracing (SKT) framework, which exploited the prerequisite and similarity relations in the knowledge structure of concepts and considered both the temporal effect on the learning sequence and the spatial effect on the knowledge structure. Despite the impressive progress, these graph-based methods performed the independent learning of exercise and answer embeddings, which may be incompatible when combined into the representations of learning interactions. Moreover, early works encoded the graph structure between exercises or knowledge concepts depending on a single type of relationship information.

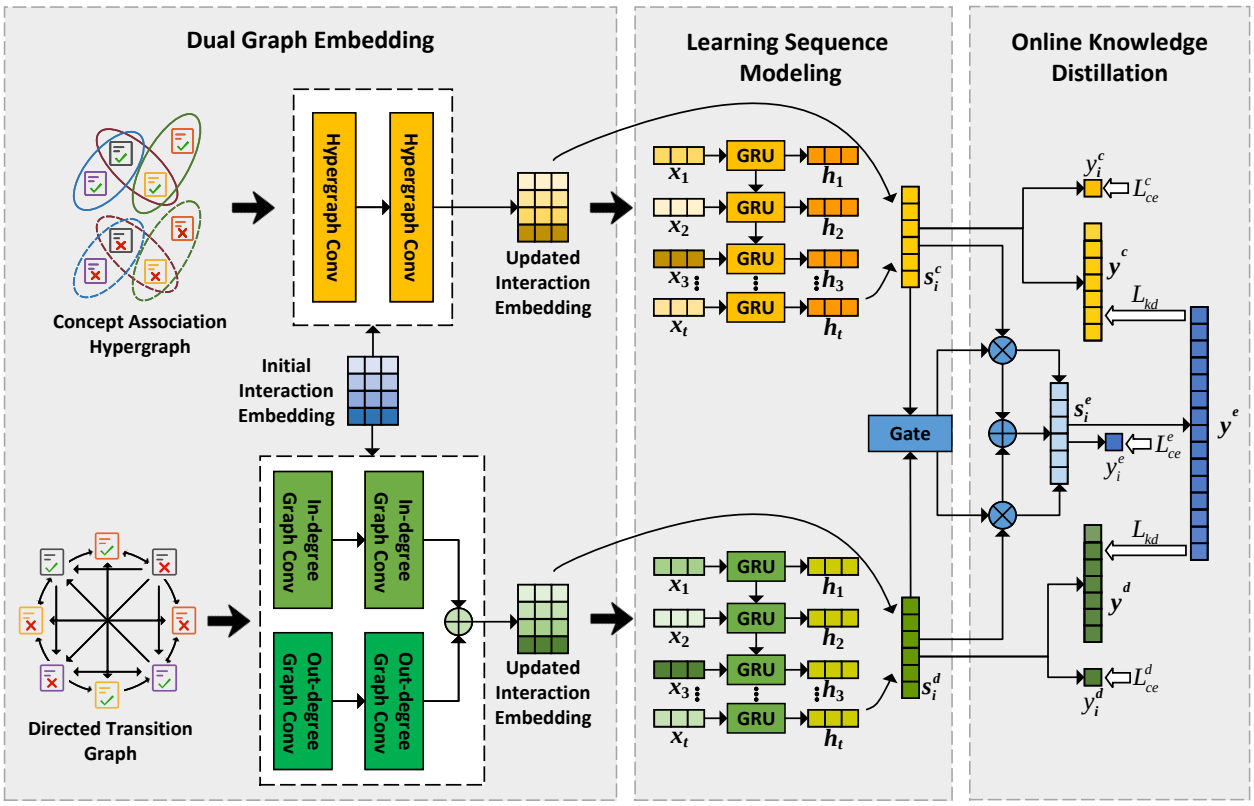


Figure 3: The overall architecture of the proposed DGEKT method.

By contrast, we establish the dual graph structure of students' learning interactions, based on which the heterogeneous exercise-concept associations and interaction transitions are jointly captured.

2.2 Graph Convolutional Networks

Graph convolutional networks (GCNs) have emerged recently as a powerful tool for analyzing non-Euclidean data structure such as social networks in the era of deep learning [33]. The basic idea of GCNs is to transform, propagate, and aggregate the information from neighboring nodes for node representation learning. Most GCNs fall into one of two families, i.e., spectral networks [13] and spatial networks [34]. The former defines convolution on graphs via the eigen decomposition of the graph Laplacian, while the latter defines graph convolution as a localized averaging operation with iteratively learned weights. In traditional GCNs, only the pairwise connections between data were considered. Hypergraph convolutional networks (HGCNs) [17] were further developed to generalize GCNS over hypergraphs that describe the topological structure involving non-pairwise relationships among nodes. On the other hand, directed graph convolutional networks (DGCNs) [18, 35] extend GCNs to directed graphs by discriminating between incoming and outgoing neighbors of each node during information propagation. In our study, building upon the observation that exercises belonging to the same knowledge concept can naturally be viewed as a collective group, we leverage HGCNs in modeling the exercise-concept associations and avoid the information loss caused by decomposing the group-wise relationships into pairwise ones. Besides, we adopt DGCNs to capture the transitions between learning interactions, which have not been fully investigated in prior research, and especially take account of the transition directions.

2.3 Knowledge Distillation

Knowledge distillation is an effective deep learning technique that has been widely used to transfer information from one network to another whilst training constructively [36]. Generally, the information is transferred in the form of response-based knowledge, feature-based knowledge, or relation-based knowledge [37]. Most knowledge distillation methods often take offline learning strategies [38], in which a teacher model needs to be first pre-trained before distillation, and it is then used to guide the training of a student model during distillation. To overcome the limitation of requiring the pre-trained teacher model, online knowledge distillation [20, 19] has been proposed to train a group

大写花体或希腊字母表示集合

Table 1: Summary of key notations and definitions.

Notation	Definition
$\mathcal{E}, \mathcal{C}$	Set of exercises and concepts
n, m	Number of exercises and concepts
$e_i \in \mathcal{E}$, $c_j \in \mathcal{C}$	Exercise and concept exemplars
$\mathcal{I}$	Exercise-answering sequence of a student
t	Number of steps in the sequence $\mathcal{I}$
$(e_k, r_k) \in \mathcal{I}$	Response r_k to exercise e_k at step k
$\mathcal{V}$	Node set consisting of all learning interactions and $\|\mathcal{V}\| = 2n$
$v_i \in \mathcal{V}, \boldsymbol{x}_i$	Node exemplar v_i and its embedding $\boldsymbol{x}_i$
$\mathcal{G}_c = (\mathcal{V}, \mathcal{E}_c)$	Concept association hypergraph (CAHG)
$\mathcal{H}_j \in \mathcal{E}_c$	Concept hyperedge containing multiple nodes
$\mathcal{G}_d = (\mathcal{V}, \mathcal{E}_d)$	Directed transition graph (DTG)
$A_{i,j}^{(\text{in})}, A_{i,j}^{(\text{out})}$	Transition probability from v_j to v_i and vice versa in DTG
$\boldsymbol{s}_i^c, \boldsymbol{s}_i^d$	Knowledge states regarding e_i of the student models from CAHG and DTG
$\boldsymbol{s}_i^e$	Knowledge state regarding e_i of the ensemble teacher model
y_i^c, y_i^d	Predicted probability of the student models for correctly answering e_i at the next step
y_i^e	Predicted probability of the ensemble teacher model for correctly answering e_i at the next step

of student models simultaneously by making them learn from each other in a peer-teaching approach. As opposed to straightforward concatenation- or adding-based fusion, we apply online knowledge distillation to ensemble the dual graph models. Besides the reference to students' responses to a single exercise at the next step, the ensemble teacher model can provide its predictions on all exercises back to student models as extra supervision, leading to more powerful modeling ability.

3 Method

In this section, we present a dual graph ensemble learning method, called DGEKT, for knowledge tracing. Firstly, we formulate the problem of knowledge tracing. Next, we detail the dual graph structure of students' learning interactions based on which the heterogeneous relationships between interactions are captured. Then, we illustrate the process of dynamic modeling of students' learning sequences. Finally, we introduce an online knowledge distillation strategy to combine the dual graph models. Fig. 3 displays the overall architecture of our method. For clarity, Table 1 summarizes some key notations and definitions used throughout this paper.

3.1 Problem Formulation

In an online learning platform, there exists a set of exercises $\mathcal{E}$. All students are asked to answer different exercises in order to acquire related knowledge. Suppose the exercise-answering sequence of a student is represented as $\mathcal{I} = \{(e_1, r_1), (e_2, r_2), \ldots, (e_t, r_t)\}$, where $e_k \in \mathcal{E}$ denotes the exercise the student answers at step k, t is the total number of steps, and r_k denotes the student's response to e_k. Similar to most previous studies on knowledge tracing, we simplify r_k as a binary value, that is, $r_k = 1$ if the student correctly answers e_k; otherwise, r_k equals zero.

Given the sequences of students' learning interactions, knowledge tracing seek to monitor students' evolving knowledge states by predicting their performance on the following exercises. Therefore, it is formulated to predict the probability that a student will give the correct response to the next exercise, i.e., $p(r_{t+1} = 1 | e_{t+1}, \mathcal{I})$.

3.2 Dual Graph Embedding

In this paper, we incorporate the potential relationship information to improve knowledge tracing, but differ from prior research [14, 15] in establishing the relational graph of learning interactions instead of exercises. Specifically, assume there are totally n exercises. $\mathcal{V} = \{v_{1+}, v_{1-}, v_{2+}, v_{2-}, \ldots, v_{n+}, v_{n-}\}$ denotes the graph nodes, in which the nodes

v_{i+} and v_{i-} represent the interactions that students correctly and incorrectly answer the exercise $e_i \in \mathcal{E}$, respectively. Obviously, the number of nodes is twice the number of exercises. v_{i+} and v_{i-} will be projected into two embeddings $\boldsymbol{x}_{i+}$ and $\boldsymbol{x}_{i-}$, which can be understood as the knowledge levels underlying students' right and wrong responses to e_i, respectively. In this way, we eliminate the necessity of having separate representations of exercises and answers in previous studies [8].

It is intuitively plausible that closely related learning interactions are more likely to reflect students' similar knowledge levels. Therefore, we establish a dual graph structure of learning interactions to capture the heterogeneous relationships between them, consisting of a concept association hypergraph (CAHG) and a directed transition graph (DTG). Then, hypergraph convolutional networks [17] and directed graph convolutional networks [18] are deployed to learn the node embeddings over CAHG and DTG, respectively. The detailed procedures are described as follows:

3.2.1 Concept Association Hypergraph

In knowledge tracing, each exercise is generally related to at least one knowledge concept $c_j \in \mathcal{C}$, where $\mathcal{C} = \{c_1, c_2, \ldots, c_m\}$ denotes the set of knowledge concepts with size m. The exercise-concept associations have been typically encoded as pairwise connections between exercises and concepts [15] or between exercises alone [14]. However, we argue that different exercises can naturally be viewed as a collective group if they are associated with the same concept, and simply decomposing the group-wise relationships into pairwise ones may cause the loss of information. From this point, we introduce the hypergraph structure CAHG to characterize the group-wise exercise-concept associations. Hypergraph is an extension of simple graph by defining a hyperedge as a subset of nodes. Let $\mathcal{G}_c = (\mathcal{V}, \mathcal{E}_c)$ denote CAHG, where $\mathcal{E}_c = \{\mathcal{H}_{1+}, \mathcal{H}_{1-}, \mathcal{H}_{2+}, \mathcal{H}_{2-}, \ldots, \mathcal{H}_{m+}, \mathcal{H}_{m-}\}$ is the set of hyperedges. Recall that each exercise corresponds to two interaction nodes of correctly and incorrectly answering the exercise. Analogously, each knowledge concept c_j is decoupled into two hyperedges $\mathcal{H}_{j+}$ and $\mathcal{H}_{j-}$, making the cardinality of $\mathcal{E}_c$ to be twice that of $\mathcal{C}$. The exercise-concept associations can be indicated by the incidence relationships between the hyperedges and their nodes. For example, if an excise e_i is associated with the concept c_j, students' right and wrong responses to e_i are probably made due to their mastery and non-mastery of c_j, leading to $v_{i+} \in \mathcal{H}_{j+}$ and $v_{i-} \in \mathcal{H}_{j-}$ at the same time. Due to the existence of correspondences between exercises and concepts, there are many-to-many correspondences between nodes and hyperedges. In the following, we shall omit the subscript $+$ or $-$ when no confusion can arise.

In the hypergraph, $\mathcal{U}_i = \{\mathcal{H}_j \,|v_i \in \mathcal{H}_j\}$ denotes the subset of hyperedges containing node v_i, based on which the degree of v_i can be defined as $d_i = |\mathcal{U}_i|$. Besides, the degree of hyperedge $\mathcal{H}_j$ is equal to the number of nodes within it, i.e., $g_j = |\mathcal{H}_j|$. To learn the embedding $\boldsymbol{x}_i$ of v_i, hypergraph convolutional networks [17] can be applied over CAHG, which defines an information propagation rule in hypergraphs via a convolution operator in each layer. Formally, the convolution operator updates $\boldsymbol{x}_i$ by aggregating the information from v_i itself as well as the local neighbors in each hyperedge to which v_i belongs:

$$\boldsymbol{x}_i^{(l)} = \phi\left(\sum_{\mathcal{H}_j \in \mathcal{U}_i} \frac{1}{g_j} \sum_{v_q \in \mathcal{H}_j} \frac{1}{\sqrt{d_i d_q}} \boldsymbol{\Theta}^{(l)} \boldsymbol{x}_q^{(l-1)}\right), \tag{1}$$

where $\boldsymbol{x}_i^{(l-1)}$ and $\boldsymbol{x}_i^{(l)}$ are the input and output embeddings of v_i in the l-th layer, $\boldsymbol{\Theta}^{(l)}$ is a learnable parameter matrix, and $\phi(\cdot)$ denotes a nonlinear activation function like LeakyReLU [39]. Intuitively, the updating smoothes the embeddings of nodes locally linked by the same hyperedges.

3.2.2 Directed Transition Graph

Except for exercise-concept associations, the rich transitions between interactions in students' learning sequences are also an important factor in knowledge tracing, which, however, has not been fully explored in prior research [14]. Intuitively, owing to the relatively short time interval between them, two consecutive interactions of students are more likely to be triggered in the knowledge states with little change. In light of this, we adopt the directed graph structure DTG for modeling the transitions between students' learning interactions. We denote DTG by $\mathcal{G}_d = (\mathcal{V}, \mathcal{E}_d)$, where the set of edges $\mathcal{E}_d \subseteq \mathcal{V} \times \mathcal{V}$ contains the edge (v_i, v_j) if the interaction v_j comes right after the interaction v_i in any student's learning sequence. Furthermore, two transition probability matrices $\boldsymbol{A}^{(\text{in})}$ and $\boldsymbol{A}^{(\text{out})}$ are defined to assign weights to edges, i.e.,

$$A_{i,j}^{(\text{in})} = \begin{cases} \dfrac{n_{j,i}}{\sum_k n_{k,i}} & \text{if } (v_j, v_i) \in \mathcal{E}_d; \\ 0 & \text{otherwise,} \end{cases} \tag{2}$$

and

$$A_{i,j}^{(\text{out})} = \begin{cases} \dfrac{n_{i,j}}{\sum_k n_{i,k}} & \text{if } (v_i, v_j) \in \mathcal{E}_d; \\ 0 & \text{otherwise}, \end{cases} \tag{3}$$

where $n_{i,j}$ denotes the number of times v_j occurs immediately after v_i in the learning sequences of all students. In this way, the historical learning records of different students can be implicitly leveraged in collaboration. We also include self-loops in DTG and set $A_{i,i}^{(\text{in})} = 1$ and $A_{i,i}^{(\text{out})} = 1$ for each node $v_i \in \mathcal{V}$. The in-degree and out-degree of v_i are computed as $d_i^{(\text{in})} = \sum_k A_{i,k}^{(\text{in})}$ and $d_i^{(\text{out})} = \sum_k A_{i,k}^{(\text{out})}$, respectively.

In DTG, we deploy directed graph convolutional networks [18, 35] for node representation learning. Particularly, we propose to update the embedding $\boldsymbol{x}_i$ of node v_i via the bi-directional information propagation in parallel: one diffuses the information along the direction of edges in the graph, so that the node embedding can be updated by receiving knowledge from the ancestor nodes:

$$\overline{\boldsymbol{x}}_i^{(l)} = \phi\left(\sum_{v_j \in \mathcal{N}_i^{(\text{in})} \cup \{v_i\}} \frac{A_{i,j}^{(\text{in})}}{\sqrt{d_i^{(\text{in})} d_j^{(\text{out})}}} \boldsymbol{\Phi}^{(l)} \boldsymbol{x}_j^{(l-1)}\right); \tag{4}$$

the other diffuses the information against the direction of edges in the graph, so that the node embedding can be updated by receiving knowledge from the descendant nodes:

$$\widetilde{\boldsymbol{x}}_i^{(l)} = \phi\left(\sum_{v_j \in \mathcal{N}_i^{(\text{out})} \cup \{v_i\}} \frac{A_{i,j}^{(\text{out})}}{\sqrt{d_i^{(\text{out})} d_j^{(\text{in})}}} \boldsymbol{\Psi}^{(l)} \boldsymbol{x}_j^{(l-1)}\right). \tag{5}$$

Here, $\mathcal{N}_i^{(\text{in})}$ and $\mathcal{N}_i^{(\text{out})}$ are the sets of ancestor and descendant nodes of v_i, respectively. $\boldsymbol{\Phi}^{(l)}$ and $\boldsymbol{\Psi}^{(l)}$ are the model parameters to be learned in the l-th layer. Finally, the output embedding $\boldsymbol{x}_i^{(l)}$ is generated by simply adding $\overline{\boldsymbol{x}}_i^{(l)}$ and $\widetilde{\boldsymbol{x}}_i^{(l)}$. More complicated combination methods can be adopted, such as fusing $\overline{\boldsymbol{x}}_i^{(l)}$ and $\widetilde{\boldsymbol{x}}_i^{(l)}$ with a learnable multilayer perceptron, but this did not lead to significant performance improvement in our experiments.

3.3 Learning Sequence Modeling

After building the dual graph structure of learning interactions, the learning sequence of a student corresponds to a unique path in each graph, which can be further represented through sequential models. In this study, we follow the well-known DKT network [8] to accomplish this process, due to its relatively simpler structure and the ability to directly handle learning interactions. Note that our method is also compatible with other recently proposed sequential modeling techniques for knowledge tracing [11, 12].

Given the learning sequence of a student $\mathcal{I} = \{(e_1, r_1), (e_2, r_2), \ldots, (e_t, r_t)\}$, each learning interaction $(e_k, r_k) \in \mathcal{I}$ can be embedded to $\boldsymbol{x}_k$ as described above. Similar to DKT using the vanilla RNN, we apply the gated recurrent unit (GRU) model to process the learning sequence. At each step k, the GRU model executes the calculations as follows:

$$\begin{aligned} \boldsymbol{r}_k &= \sigma\left(\boldsymbol{W}_r\left[\boldsymbol{h}_{k-1}, \boldsymbol{x}_k\right] + \mathbf{b}_r\right), \\ \boldsymbol{u}_k &= \sigma\left(\boldsymbol{W}_u\left[\boldsymbol{h}_{k-1}, \boldsymbol{x}_k\right] + \mathbf{b}_u\right), \\ \boldsymbol{c}_k &= \tanh\left(\boldsymbol{W}_c\left[\boldsymbol{r}_k \odot \boldsymbol{h}_{k-1}, \boldsymbol{x}_k\right] + \mathbf{b}_c\right), \\ \boldsymbol{h}_k &= (\mathbf{1} - \boldsymbol{u}_k) \odot \boldsymbol{h}_{k-1} + \boldsymbol{u}_k \odot \boldsymbol{c}_k, \end{aligned} \tag{6}$$

where $\boldsymbol{h}_{k-1}$ is a hidden state summarizing the information of past steps. $\boldsymbol{h}_{k-1}$ and $\boldsymbol{x}_k$ are concatenated and transformed into a reset gate $\boldsymbol{r}_k$ and an update gate $\boldsymbol{u}_k$, respectively. The former determines how much the past information should be forgotten, whereas the latter controls how much the information needs to be brought into the current hidden state $\boldsymbol{h}_k$. Then, $\boldsymbol{h}_{k-1}$ is reset with $\boldsymbol{r}_k$, and again concatenated with $\boldsymbol{x}_k$ to generate a memory cell $\boldsymbol{c}_k$. $\boldsymbol{c}_k$ represents the new information to be added to $\boldsymbol{h}_k$. Finally, $\boldsymbol{h}_k$ is computed as the combination of $\boldsymbol{h}_{k-1}$ and $\boldsymbol{c}_k$, and $\boldsymbol{u}_k$ serves as a balance factor in this procedure. In Eq. (6), $\mathbf{W}_r$, $\mathbf{W}_u$, and $\mathbf{W}_c$ are the transformation matrices, and $\mathbf{b}_r$, $\mathbf{b}_u$, and $\mathbf{b}_c$ are the bias terms to be learned. $\odot$ denotes the Hadamard product, and $\sigma(\cdot)$ and $\tanh(\cdot)$ denote the sigmoid and tanh activation functions, respectively.

In DKT, the latest hidden state $\boldsymbol{h}_t$ is considered to reflect a student's knowledge state. Given an exercise $e_i \in \mathcal{E}$, we further concatenate $\boldsymbol{h}_t$ with the node embeddings $\boldsymbol{x}_{i+}$ and $\boldsymbol{x}_{i-}$ to represent the student's knowledge state regarding e_i:

$$\boldsymbol{s}_i = \left[\boldsymbol{h}_t, \boldsymbol{x}_{i+}, \boldsymbol{x}_{i-}\right], \tag{7}$$

which comprehensively considers $\boldsymbol{h}_t$ and the knowledge levels for answering e_i correctly and incorrectly. $\boldsymbol{s}_i$ is then mapped to the probability that the student will give the correct response to e_i at the next step $t+1$:

$$y_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}, \tag{8}$$

where

$$z_i = \boldsymbol{w}_o{}^T \boldsymbol{s}_i + b_o \tag{9}$$

is the logit or unnormalized log probability [40], and $\boldsymbol{w}_o$ and b_o are the readout weight and bias parameters, respectively. To jointly learn all model parameters, a cross-entropy loss between the predicted probability y_i and the true response r_{t+1} can be optimized:

$$\mathcal{L}_{ce} = -\sum_{i=1}^{n} \delta_{t+1}^{i}(r_{t+1}\log y_i + (1-r_{t+1})\log(1-y_i)), \tag{10}$$

where δ_{t+1}^{i} is a Dirac delta returning 1 when e_i is the exercise the student actually did at the next step, and 0 otherwise.

3.4 Online Knowledge Distillation

Following the above procedures, we can separately model the learning sequence of a student with the node embeddings of CAHG and DTG, and obtain two independent representations of the knowledge states $\boldsymbol{s}_i^c$ and $\boldsymbol{s}_i^d$ as well as the predicted probabilities y_i^c and y_i^d for the exercise e_i. To ensemble the two graph models, a natural thought is to directly combine $\boldsymbol{s}_i^c$ and $\boldsymbol{s}_i^d$ or y_i^c and y_i^d [41, 42]. However, in this study, we introduce the idea of online knowledge distillation [19] for this purpose. As shown in Eq. (10), although the knowledge tracing model is expected to predict the students' responses to the exercises related to different concepts, it is optimized merely with respect to the prediction accuracy on a single exercise at the next step. In light of this, we regard the two graph models as peer student models and ensemble them to build a stronger teacher model, which provides its predicted results on all exercises back to student models as extra supervision.

Specifically, due to the importance variety of student models, we adopt a gating mechanism [20] to adaptively ensemble their generated knowledge states $\boldsymbol{s}_i^c$ and $\boldsymbol{s}_i^d$ to build the teacher model:

$$\boldsymbol{s}_i^e = \boldsymbol{g} \odot \boldsymbol{s}_i^c + (\mathbf{1} - \boldsymbol{g}) \odot \boldsymbol{s}_i^d, \tag{11}$$

where the gate $\boldsymbol{g}$ is given by

$$\boldsymbol{g} = \sigma\left(\boldsymbol{W}_g\left[\boldsymbol{s}_i^c, \boldsymbol{s}_i^d\right] + \mathbf{b}_g\right) \tag{12}$$

with $\mathbf{W}_g$ and $\mathbf{b}_g$ being the learnable parameters. Intuitively, $\boldsymbol{g}$ serves to balance the importance of the two student models. Given $\boldsymbol{s}_i^c$, $\boldsymbol{s}_i^d$, and $\boldsymbol{s}_i^e$, their corresponding logits for the exercise e_i can be yield from Eq. (9) and denoted by z_i^c, z_i^d, and z_i^e, respectively.

To distill the knowledge of the ensemble teacher model back into student models, we firstly compute the soft version of their predicted probabilities at a temperature of γ:

$$\begin{aligned} \tilde{y}_i^c &= \frac{1}{1+e^{-z_i^c/\gamma}}, \\ \tilde{y}_i^d &= \frac{1}{1+e^{-z_i^d/\gamma}}, \\ \tilde{y}_i^e &= \frac{1}{1+e^{-z_i^e/\gamma}}. \end{aligned} \tag{13}$$

Then, we design a distillation loss to encourage each student model matching with the teacher's predictions across all exercises:

$$\mathcal{L}_{kd} = \frac{1}{n}\sum_{i=1}^{n} \|\tilde{y}_i^e - \tilde{y}_i^c\|_1 + \left\|\tilde{y}_i^e - \tilde{y}_i^d\right\|_1. \tag{14}$$

Here, the discrepancy between the student and teacher models is measured by the L_1-norm distance of their outputs rather than the Kullback-Leibler (KL) divergence commonly used in knowledge distillation [38]. This is supported by the recent finding that the L_1-norm distance is a tighter bound on the classification error probability [43]. Moreover, the distillation loss can also be viewed as a learned label smoothing regularization [44, 45], which makes the student and teacher models regularize the training process of each other and therefore improves model generalization.

Table 2: Dataset statistics.

Dataset	ASSIST09	ASSIST17	EdNet
# Students	4,151	1,709	5000
# Exercises	16,891	3,162	10,795
# Concepts	101	102	188
# Exercises per concept	167.24	37.98	148.73
# Concepts per exercise	1.00	1.23	2.26
# Records	274,590	942,816	225,625
% Correct answering records	66.16%	37.26%	58.99%
% Incorrect answering records	33.84%	62.74%	41.01%

Finally, both the student and teacher models need to be trained simultaneously with reference to the prediction performance of the student's response to the next exercise, according to the cross-entropy loss defined in Eq. (10). Therefore, we obtain the overall loss for training the proposed DGEKT as:

$$\mathcal{L} = \mathcal{L}_{ce}^{c} + \mathcal{L}_{ce}^{d} + \mathcal{L}_{ce}^{e} + \lambda \mathcal{L}_{kd}, \tag{15}$$

where $\mathcal{L}_{ce}^{c}$ and $\mathcal{L}_{ce}^{d}$ are the cross-entropy loss terms for the student models derived from CAHG and DTG, respectively, and $\mathcal{L}_{ce}^{e}$ is that for the ensemble teacher model. λ is a trade-off hyperparameter controlling the relative contributions of the cross-entropy and distillation losses. Once the training is completed, the ensemble teacher model is deployed for achieving higher accuracy during testing.

4 Experiment

In this section, we describe the experimental setup details and report a series of experimental results to validate the effectiveness of the proposed DGEKT.

4.1 Dataset

We evaluate our method on three benchmark datasets for knowledge tracing, i.e., ASSIST09, ASSIST17, and EdNet. The statistics of the three datasets are listed in Table 2. On all datasets, the maximum length of students' learning sequences is set to 50. If a student has more than 50 exercise-answering records, we partition his/her learning sequence into multiple subsequences. We randomly select 80% of students for training and validation and the remaining 20% for testing.

ASSIST09[2]

The dataset is collected from the ASSISTments online tutoring platform[3] during the 2009-2010 school year. We filter out the exercises without knowledge concepts and the students with less than three learning interactions.

ASSIST17[4]

The dataset is obtained from the ASSISTments data mining competition 2017. We do the same preprocessing as ASSIST09.

EdNet[5]

The dataset is contributed by Choi et al. in [46]. It is a large dataset with over 130 million records that involve about 0.78 million students. For computational efficiency, we sample the learning records of 5,000 students from the dataset.

[2]https://sites.google.com/site/assistmentsdata/home/2009-2010-assistment-data
[3]https://new.assistments.org
[4]https://sites.google.com/view/assistmentsdatamining/data-mining-competition-2017
[5]https://github.com/riiid/ednet

4.2 Evaluation Metric

The prediction of students' responses to exercises is considered as a binary classification task, i.e., predicting whether a student answers an exercise correctly or not. As shown in Table 2, on account of the imbalance between the correct and incorrect answering records, we evaluate the classification performance of each algorithm in terms of the Area Under ROC Curve (AUC). A higher value of AUC indicates better performance in predicting students' responses, and the value of 0.5 represents the performance of making predictions by random guessing.

4.3 Competitor

可以参考以下将对比算法分类

We compare the performance of DGEKT against eight knowledge tracing models in three categories, including:

4.3.1 Traditional Methods

These methods adopt non-deep learning techniques to predict students' responses.

- **BKT** [6] is based on the hidden Markov model, and models students' knowledge states for each concept as a binary variable.
- **KTM** [24] uses the factorization machine to encode the side information about exercises or students into the model. For fairness, we only consider the exercise IDs and concept IDs as side information in comparison.

4.3.2 Sequential Modeling Methods

These methods address knowledge tracing with deep sequence models.

- **DKT** [8] applies RNN to process the learning sequences of students, and represents their knowledge states with the hidden states of RNN.
- **DKVMN** [9] introduces a memory-augmented neural network, which uses a key matrix to represent knowledge concepts and a value matrix to store and update the mastery of corresponding concepts of students over time.
- **SAKT** [11] employs the self-attention mechanism to identify the past learning interactions relevant to the exercise to be answered, and makes the prediction by focusing on these relevant ones.
- **AKT** [12] uses the transformer network to model the learning sequences of students, and proposes a monotonic attention mechanism using an exponential decay curve to reduce the importance of interactions in the distant past.

4.3.3 Graph-based Methods

These methods incorporate the graph structure of exercises or knowledge concepts as a relational inductive bias to improve knowledge tracing.

- **GKT** [14] learns the relationships between knowledge concepts and formulates knowledge tracing as a time-series node-level classification problem with GCNs.
- **PEBG** [15] constructs the bipartite graph of exercise-concept relations to learn the pre-training embeddings of exercises, which are then used as the inputs to the DKT model. For fairness, the attribute features of exercises defined in [15], such as difficulty level, type, and average response time, were not exploited in our implementation.
- **DGEKT** is our proposed method in this paper.

4.4 Implementation Details

Our implementation determined the optimal hyperparameters of baseline methods either following the suggested settings in the original papers or based on the AUC value obtained on the validation set. In the proposed DGEKT, all learning interactions were consistently embedded in a 256-dimensional space. We adopted a two-layered graph network for node embedding on CAHG and DTG, respectively. The number of hidden layers and the number of cells per layer in the GRU model were 1 and 128, respectively. For online knowledge distillation, we set the temperature factor $\gamma = 0.5$ and the trade-off hyperparameter $\lambda = 0.01$, respectively. We will discuss the sensitivity of DGEKT with regard to the changing of different hyperparameters later.

Table 3: Performance comparison on three benchmark datasets.

Category	Method	ASSIST09	ASSIST17	EdNet
Traditional	BKT [6]	72.05%	63.65%	66.68%
	KTM [24]	72.92%	67.32%	<u>69.79%</u>
Sequential Modeling	DKT [8]	73.37%	<u>70.70%</u>	67.08%
	DKVMN [9]	73.98%	66.70%	66.75%
	SAKT [11]	74.98%	66.48%	63.45%
	AKT [12]	<u>75.61%</u>	70.34%	69.55%
Graph-based	GKT [14]	73.96%	69.16%	67.89%
	PEBG [15]	73.47%	68.03%	66.94%
	DGEKT	**76.56%**	**77.54%**	**70.07%**

The best result in terms of each metric is indicated in bold, and the second best one is underlined. This convention is also adopted in the following tables.

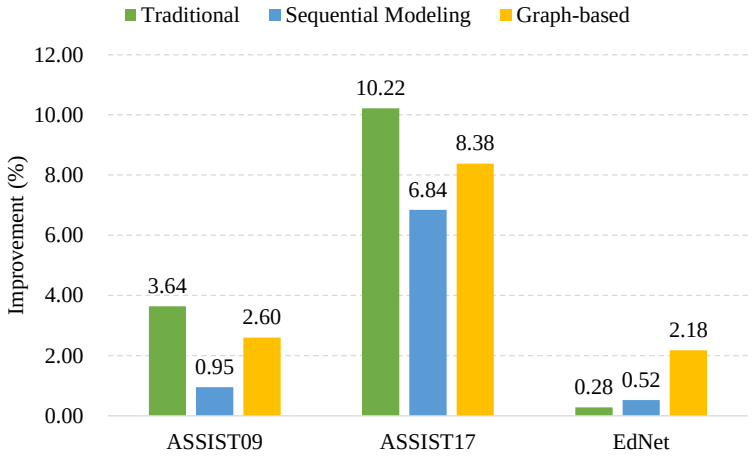


Figure 4: The improvement over baselines of different categories.

We implemented the network training and testing using the deep learning library Pytorch[6]. The network was trained with the mini-batch Adam optimizer [47]. We set the batch size to 128. The initial learning rate was 10^{-3} for all layers, and the maximum number of epochs was 600 during training. For the sake of reproducibility, the codes and models of our work have been released at `https://github.com/Yumo216/DGEKT`.

4.5 Performance Comparison

Table 3 lists the performance of the proposed DGEKT compared to baseline methods. We can observe that DGEKT outperforms all of the eight baselines and achieves the best performance on all datasets. More precisely, it is better than the best baseline by 0.95%, 6.84%, and 0.28% in terms of AUC on ASSIST09, ASSIST17, and EdNet, respectively. It deserves to mention that DGEKT is equipped with a sequential modeling component similar to DKT, but it is significantly ahead of DKT. In addition, even without the attention mechanism [27] in sequential modeling, the performance of DGEKT is still consistently higher than the attention-based baselines, such as SAKT and AKT. That means that introducing the modules of dual graph embedding and online knowledge distillation into DGEKT well benefits the performance of knowledge tracing. This point will be further verified later.

Overall, traditional methods frequently fall behind recently proposed deep learning-based methods, including sequential modeling and graph-based baselines. This highlights the merit of the end-to-end learning techniques for knowledge tracing. However, traditional methods can achieve comparable or even better performance in some cases. For example, KTM yields the second-best AUC score on EdNet. Besides, one can see that although DKT has a very simple network structure, there is no constant winner between it and those more complex sequential modeling contenders, i.e., DKVMN,

[6]https://pytorch.org

Table 4: Setting differences between variant methods.

Variant Method	Hypergraph	Directed Graph	CAHG	DTG	Knowledge Distillation
CAG	×	✓	✓	✓	✓
TG	✓	×	✓	✓	✓
RmCAHG	×	✓	×	✓	✓
RmDTG	✓	×	✓	×	✓
RmOKD	✓	✓	✓	✓	×

Table 5: Performance comparison between DGEKT and its variant methods.

Method	ASSIST09	ASSIST17	EdNet
CAG	74.77%	77.14%	69.98%
TG	73.68%	**77.67%**	69.71%
RmCAHG	73.02%	76.00%	69.72%
RmDTG	76.27%	76.76%	69.28%
RmOKD	76.34%	76.89%	69.40%
DGEKT	**76.56%**	77.54%	**70.07%**

SAKT, and AKT. Therefore, it is believed that more skillful techniques should be developed in modeling the learning sequences of students for knowledge tracing.

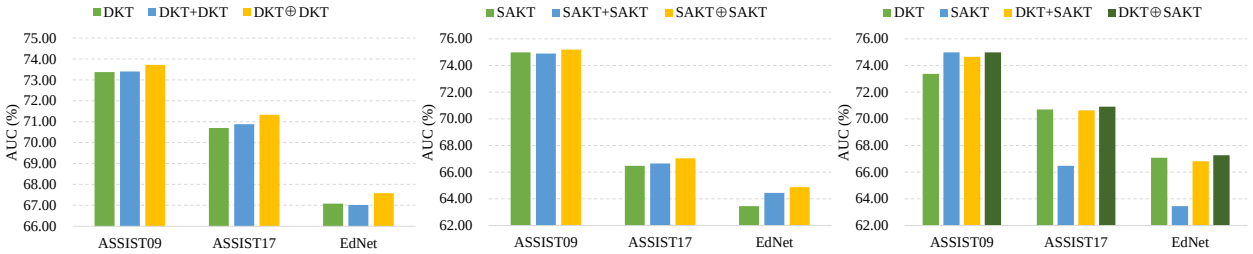
In Fig. 4, we also report the improvement of DGEKT over the best baseline in different categories. Notably, while they all leverage the relationship information between exercises, DGEKT is substantially superior to the other two graph-based methods, i.e., GKT and PEBG, resulting in at least 2% improvement on the three benchmark datasets. The main reason may be attributed to the fact that in addition to exercise-concept associations, DGEKT also exploits the transitions between students' learning interactions, which are not fully explored in previous studies. Moreover, it can be clearly seen that DGEKT exhibits a greater advantage on ASSIST17. For example, the minimum improvement arises from 0.95% and 0.28% on ASSIST09 and EdNet to 6.84% on ASSIST17, respectively. We conjecture that this is because ASSIST17 contains more abundant exercise-answering records as listed in Table 2, from which DGEKT can establish a more dense directed transition graph of learning interactions and lead to better performance.

4.6 Ablation Study

To further investigate the contributions of different components of DGEKT, we carried out a series of ablation studies by devising five variant methods of DGEKT:

- **CAG** simplifies DGEKT via adopting a simple graph rather than a hypergraph in modeling the exercise-concept associations. In CAG, any pair of learning interactions are connected if they are associated with the same concept.
- **TG** simplifies DGEKT by using the undirected version of GCNs for modeling the transitions between learning interactions. It creates an undirected edge for each pair of consecutive interactions in students' learning sequences.
- **RmCAHG** removes the concept association hypergraph from DGEKT, meaning that only the transitions between learning interactions are leveraged to learn their embeddings.
- **RmDTG** removes the directed transition graph from DGEKT, meaning that only the exercise-concept associations are leveraged to learn the embeddings of learning interactions.
- **RmOKD** removes the procedure of online knowledge distillation in DGEKT. Instead, it concatenates the two representations of knowledge states generated by the dual graph models, and maps the result into the predicted probability via a sigmoid classifier layer.

Table 4 summarizes the difference in the settings of these variant methods. We show the performance comparison between them and DGEKT in Table 5, from which the following observations can be made:



(a) DKT (b) SAKT (c) DKT and SAKT

Figure 5: Performance comparison between the concatenation scheme and online knowledge distillation for model ensemble in knowledge tracing. The symbols $+$ and $\oplus$ indicate the concatenation and online knowledge distillation based model ensemble, respectively.

4.6.1 Rationality of Graph Type

DGEKT models the exercise-concept associations and learning interaction transitions using hypergraphs and directed graphs, respectively. When changing the graph types to simple undirected graphs, both CAG and TG experience performance degradation in most cases. For example, CAG and TG are substantially inferior to DGEKT by 1.79% and 2.88% on ASSIST09, respectively. The only exception is TG on ASSIST17, which is slightly better than DGEKT. As aforementioned, this is perhaps because a more dense transition graph can be built on ASSIST17, leading to an ambiguous distinction between the directed and undirected graph settings. The above observations confirm our belief that simply compressing the group-wise exercise-concept associations into pairwise ones could cause the loss of information. At the same time, if the edge direction is ignored when diffusing the information along transition paths, it may deteriorate the performance.

4.6.2 Contribution of Dual Graph Structure

DGEKT integrates the dual graph structure consisting of CAHG and DTG to capture the heterogeneous relationships between learning interactions. When removing either CAHG or DTG, RmCAHG and RmDTG are worse than DGEKT. Besides, even without online knowledge distillation, RmOKD can usually obtain higher performance over RmCAHG and RmDTG by simply combining their outputs. The results suggest that CAHG and DTG complement each other, and the dual graph structure is beneficial to improving knowledge tracing.

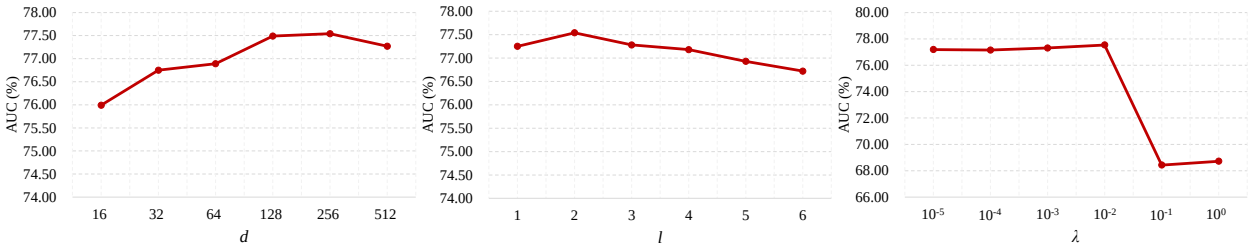
4.6.3 Benefit from Knowledge Distillation

In DGEKT, we introduce the idea of online knowledge distillation to combine the dual graph models. As demonstrated in Table 5, DGEKT consistently outperforms RmOKD on all datasets. Also, we believe that online knowledge distillation can emerge as a highly effective solution for model ensemble in knowledge tracing. To clarify this point, we separately adopt the concatenation scheme and online knowledge distillation to ensemble two DKT models, two SAKT models, as well as a DKT and a SAKT model. The performance of the resulting ensemble models is displayed in Fig. 5, where the symbols $+$ and $\oplus$ indicate the concatenation and online knowledge distillation based ensemble, respectively. Overall, the concatenation-based fusion may fail to produce extra performance gains, especially when ensembling the models with different architectures. For example, DKT+SAKT obviously falls behind the better one of DKT and SAKT. On the contrary, the strategy of online knowledge distillation offers a stable improvement over the single model, no matter through the homogeneous ensemble models (i.e., DKT$\oplus$DKT and SAKT$\oplus$SAKT) or the heterogeneous one (i.e., DKT$\oplus$SAKT).

4.7 Sensitivity Analysis

In order to investigate the sensitivity of DGEKT, we evaluate the impact of key hyperparameters on its performance. Specifically, we consider the hyperparameters of embedding size d and depth l in the dual graph structure, as well as the trade-off controller λ between the cross-entropy and distillation losses. The experiments were carried out on ASSIST17, and the results are shown in Fig. 6.

For the graph embedding size d, we impose the simplified assumption that different graph layers output the same size of node embeddings to reduce the complexity. The value of d is restricted to be a power of 2, ranging from 16 to 512. As can be seen in Fig. 6(a), the performance of DGEKT improves gradually with the increase of d, but overly



(a) Graph embedding size (b) Graph network depth (c) Trade-off controller

Figure 6: Impact of different hyperparameters.

increasing the embedding size could stagnate or even deteriorate the performance. The best performance is achieved when $d = 256$. For the graph network depth l, we simply assume that CAHG and DTG have the same number of layers. Fig. 6(b) illustrates how the change of l affects the performance of DGEKT. We can observe that the performance goes down gradually when l is larger than 2. The phenomenon is mainly caused by the over-smoothing issue [48] that the node representations become mixed and indistinguishable from each other after many layers of message passing in graph neural networks. For the trade-off controller λ, we perform a logarithmic grid search from 10^{-5} to 10^{0} with a scaling factor of 10, as shown in Fig. 6(c). DGEKT keeps relatively steady performance when λ is between 10^{-5} and 10^{-2}, but rapidly declines with a further increase of λ. On the whole, there is no drastic fluctuation in the performance of DGEKT when different hyperparameters vary within a wide range. Therefore, it can be said that DGEKT has good model stability, and the optimal values of its hyperparameters are easily determined.

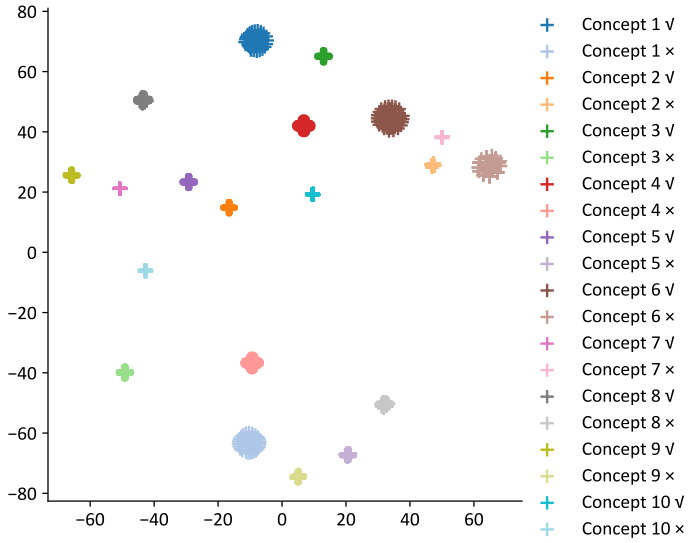
4.8 Model Visualization

In order to better understand the ability of DGEKT to represent students' learning interactions, we visualize the embeddings of learning interactions generated from CAHG and DTG.

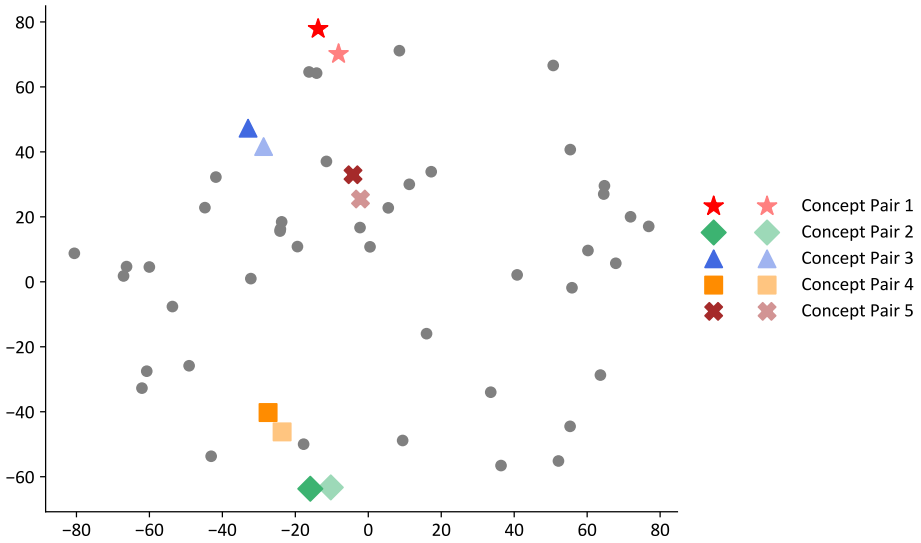
Firstly, we randomly chose ten knowledge concepts, each of which has been decoupled into two hyperedges in CAHG, leading to a total of 20 hyperedges. Then, the embeddings of learning interactions belonging to these hyperedges were projected into a two-dimensional space using the t-SNE algorithm [49]. Fig. 7(a) displays the projection results. As can be seen, learning interactions associated with different knowledge concepts can be well separated. Meanwhile, we notice that the correct and incorrect responses to the same concept are usually distributed far away from each other. The phenomenon is reasonable in the sense that correct and incorrect responses to the same concept somewhat reflect two opposite knowledge states, i.e., the mastery and non-mastery of the concept. From these results, we can conclude that the information of exercise-concept associations is well integrated into our DGEKT model.

Besides, we find that DGEKT has the potential to discover the latent structure of knowledge concepts [8]. In our datasets, due to the lack of descriptions of knowledge concepts, the dependencies between concepts cannot be directly perceived. Instead, we estimate the dependencies between two concepts by counting how many times they are associated with the same exercise. Intuitively, if two concepts frequently appear together in the same exercise, they are likely to have the prerequisite or similarity relation in learning and education. Fig. 7(b) shows the two-dimensional projections of 50 randomly sampled concepts. Each concept is represented by the embedding center of learning interactions associated with the concept in CAHG. In the figure, five pairs of concepts with high co-occurrence frequency are also indicated by different colored markers. It can be clearly observed that the two concepts in each pair are embedded close to each other. This suggests that we may infer the dependencies between knowledge concepts through the spatial distribution of their CAHG embeddings.

For DTG, we clustered all embeddings of learning interactions into 20 groups and randomly picked out ten students to exhibit their historical learning sequences. In Fig. 8, the students' learning interactions are labeled by their respective clusters and distinguished by colors, and each row represents the learning sequence of a student. It can be seen that consecutive interactions in a learning sequence are frequently partitioned into the same cluster, implying that they have approximate embeddings generated by DTG. The finding demonstrates that DGEKT indeed captures the transitions between learning interactions across students with DTG.



(a) CAHG embeddings of learning interactions associated with ten knowledge concepts.



(b) CAHG embeddings of 50 randomly sampled knowledge concepts, among which five pairs of concepts with high co-occurrence frequency are indicated by different colored markers.

Figure 7: Visualization of the learning interaction embeddings generated from CAHG.

5 Conclusion

In this paper, we investigate the dual graph structure of students' learning interactions that encodes the heterogeneous exercise-concept associations and interaction transitions. In particular, we propose a dual graph ensemble learning method, named DGEKT, for knowledge tracing. DGEKT integrates the two types of relationship information with hypergraph and directed graph convolutional networks. To ensemble the dual graph models, we introduce the technique of online knowledge distillation rather than the straightforward concatenation- or adding-based fusion. DGEKT leverages online knowledge distillation to form an ensemble teacher model from the dual graph models, which offers its predictions on all exercises as extra supervision for more powerful modeling capacity. We performed a comparative study with eight previous methods on three benchmark datasets and verified the effectiveness of DGEKT for knowledge tracing. We also demonstrated that the modules of dual graph learning and online knowledge distillation indeed benefit the performance of knowledge tracing. A sensitivity analysis was performed to show that DGEKT has good model stability, and a visualization study was conducted to provide more insights into DGEKT.

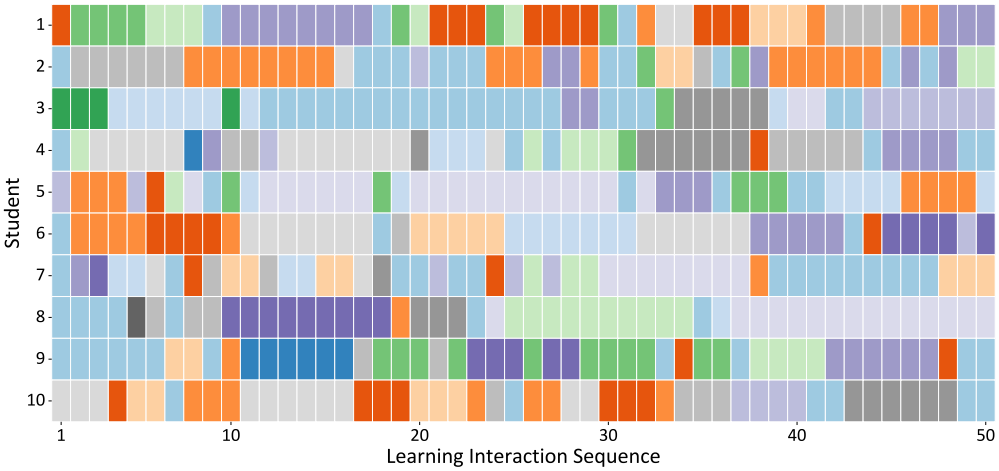


Figure 8: Visualization of the learning interaction sequences of ten students. All learning interactions are clustered into 20 groups based on DTG embeddings and distinguished by colors.

Building upon the current study, our future work will be carried out in three directions. Firstly, we intend to explore more elaborate network architectures instead of vanilla DKT to better model students' learning sequences. In the experiments, we have noticed that the attention-based sequential modeling methods have no significant advantage over DKT. It is an important issue to study how to make the attention mechanism play a more effective role in modeling learning sequences. Besides, most exiting knowledge tracing methods only estimate the binary response (i.e., correctness or incorrectness) of students to exercises, but not all incorrect responses are equal: there can be numerous incorrect ways to answer an exercise caused by different underlying errors [50]. Based on the recently released dataset [51] that contains the specific options students select on multiple choice exercises, we plan to extend our method beyond correctness prediction to the task of predicting students' option selections. Finally, as pointed out in [52], students may have close knowledge states if they behave similarly in learning. Therefore, we will investigate how to measure the similarity between students' learning behaviors and explicitly leverage the inter-student information to enhance knowledge tracing.

Acknowledgments

This work was partially supported by the National Natural Science Foundation of China (Grant No. 62077033 and No. 62177031), the Shandong Provincial Natural Science Foundation Key Project (Grant No. ZR2020KF015), the Shandong Provincial Natural Science Foundation (Grant No. ZR2021MF044) and the Fostering Project of Dominant Discipline and Talent Team of Shandong Province Higher Education Institutions.

References

[1] Zhenya Huang, Qi Liu, Yuying Chen, Le Wu, Keli Xiao, Enhong Chen, Haiping Ma, and Guoping Hu. Learning or forgetting? a dynamic approach for tracking the knowledge proficiency of students. *ACM Transactions on Information Systems*, 38(2):1–33, 2020.

[2] Chao Wang, Hengshu Zhu, Peng Wang, Chen Zhu, Xi Zhang, Enhong Chen, and Hui Xiong. Personalized and explainable employee training course recommendations: A bayesian variational approach. *ACM Transactions on Information Systems*, 40(4), 2021.

[3] Wayne Xin Zhao, Wenhui Zhang, Yulan He, Xing Xie, and Ji-Rong Wen. Automatically learning topics and difficulty levels of problems in online judge systems. *ACM Transactions on Information Systems*, 36(3), 2018.

[4] Jian Zong, Chaoran Cui, Yuling Ma, Li Yao, Meng Chen, and Yilong Yin. Behavior-driven student performance prediction with tri-branch convolutional neural network. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pages 2353–2356, 2020.

[5] Qi Liu, Shuanghong Shen, Zhenya Huang, Enhong Chen, and Yonghe Zheng. A survey of knowledge tracing. *arXiv preprint arXiv:2105.15106*, 2021.

[6] Albert T Corbett and John R Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4):253–278, 1995.

[7] Mohammad Khajah, Robert V Lindsey, and Michael C Mozer. How deep is knowledge tracing? *Proceedings of the 9th International Conference on Educational Data Mining*, pages 94–101, 2016.

[8] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[9] Jiani Zhang, Xingjian Shi, Irwin King, and Dit-Yan Yeung. Dynamic key-value memory networks for knowledge tracing. In *Proceedings of the 26th International Conference on World Wide Web*, pages 765–774, 2017.

[10] Ghodai Abdelrahman and Qing Wang. Knowledge tracing with sequential key-value memory networks. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 175–184, 2019.

[11] Shalini Pandey and George Karypis. A self-attentive model for knowledge tracing. In *Proceedings of The 12th International Conference on Educational Data Mining*, pages 384–389, 2019.

[12] Aritra Ghosh, Neil Heffernan, and Andrew S Lan. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2330–2339, 2020.

[13] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, 2017.

[14] Hiromi Nakagawa, Yusuke Iwasawa, and Yutaka Matsuo. Graph-based knowledge tracing: Modeling student proficiency using graph neural network. In *Proceedings of the IEEE/WIC/ACM International Conference on Web Intelligence*, pages 156–163, 2019.

[15] Yunfei Liu, Yang Yang, Xianyu Chen, Jian Shen, Haifeng Zhang, and Yong Yu. Improving knowledge tracing via pre-training question embeddings. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence*, pages 1556–1662, 2021.

[16] Xu Chen, Kun Xiong, Yongfeng Zhang, Long Xia, Dawei Yin, and Jimmy Xiangji Huang. Neural feature-aware recommendation with signed hypergraph convolutional network. *ACM Transactions on Information Systems*, 39(1), 2020.

[17] Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. Hypergraph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 3558–3565, 2019.

[18] Lei Shi, Yifan Zhang, Jian Cheng, and Hanqing Lu. Skeleton-based action recognition with directed graph neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7912–7921, 2019.

[19] Ying Zhang, Tao Xiang, Timothy M Hospedales, and Huchuan Lu. Deep mutual learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4320–4328, 2018.

[20] Xu Lan, Xiatian Zhu, and Shaogang Gong. Knowledge distillation by on-the-fly native ensemble. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[21] Shuanghong Shen, Zhenya Huang, Qi Liu, Yu Su, Shijin Wang, and Enhong Chen. Assessing student's dynamic knowledge state by exploring the question difficulty effect. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 427–437, 2022.

[22] Tanja Käser, Severin Klingler, Alexander G Schwing, and Markus Gross. Dynamic bayesian networks for student modeling. *IEEE Transactions on Learning Technologies*, 10(4):450–462, 2017.

[23] Philip I. Pavlik, Hao Cen, and Kenneth R. Koedinger. Performance factors analysis –a new alternative to knowledge tracing. In *Proceedings of the 2009 Conference on Artificial Intelligence in Education*, pages 531–538, 2009.

[24] Jill-Jênn Vie and Hisashi Kashima. Knowledge tracing machines: Factorization machines for knowledge tracing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 750–757, 2019.

[25] Chong Chen, Min Zhang, Yongfeng Zhang, Yiqun Liu, and Shaoping Ma. Efficient neural matrix factorization without sampling for recommendation. *ACM Transactions on Information Systems*, 38(2), 2020.

[26] Jing Li, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Tao Lian, and Jun Ma. Neural attentive session-based recommendation. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, pages 1419–1428, 2017.

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[28] Qi Liu, Zhenya Huang, Yu Yin, Enhong Chen, Hui Xiong, Yu Su, and Guoping Hu. Ekt: Exercise-aware knowledge tracing for student performance prediction. *IEEE Transactions on Knowledge and Data Engineering*, 33(1):100–115, 2019.

[29] Chenyang Wang, Weizhi Ma, Min Zhang, Chuancheng Lv, Fengyuan Wan, Huijie Lin, Taoran Tang, Yiqun Liu, and Shaoping Ma. Temporal cross-effects in knowledge tracing. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, pages 517–525, 2021.

[30] Ting Long, Yunfei Liu, Jian Shen, Weinan Zhang, and Yong Yu. Tracing knowledge state with individual cognition and acquisition estimation. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 173–182, 2021.

[31] Tianqi Wang, Fenglong Ma, and Jing Gao. Deep hierarchical knowledge tracing. In *Proceedings of The 12th International Conference on Educational Data Mining*, pages 671–674, 2019.

[32] Shiwei Tong, Qi Liu, Wei Huang, Zhenya Huang, Enhong Chen, Chuanren Liu, Haiping Ma, and Shijin Wang. Structure-based knowledge tracing: An influence propagation view. In *Proceedings of the 36th International Conference on Data Mining*, pages 541–550, 2020.

[33] Minghao Zhao, Qilin Deng, Kai Wang, Runze Wu, Jianrong Tao, Changjie Fan, Liang Chen, and Peng Cui. Bilateral filtering graph convolutional network for multi-relational social recommendation in the power-law networks. *ACM Transactions on Information Systems*, 40(2), 2021.

[34] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[35] Xitong Zhang, Yixuan He, Nathan Brugnone, Michael Perlmutter, and Matthew Hirn. Magnet: A neural network for directed graphs. *Advances in Neural Information Processing Systems*, 34:27003–27015, 2021.

[36] Lin Wang and Kuk-Jin Yoon. Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021. doi: 10.1109/TPAMI.2021.3055564.

[37] Jianping Gou, Baosheng Yu, Stephen J Maybank, and Dacheng Tao. Knowledge distillation: A survey. *International Journal of Computer Vision*, 129(6):1789–1819, 2021.

[38] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[39] Andrew L Maas, Awni Y Hannun, Andrew Y Ng, et al. Rectifier nonlinearities improve neural network acoustic models. In *Proceedings of the 30th International Conference on Machine Learning*, volume 30, page 3, 2013.

[40] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016.

[41] Xin Xia, Hongzhi Yin, Junliang Yu, Qinyong Wang, Lizhen Cui, and Xiangliang Zhang. Self-supervised hypergraph convolutional networks for session-based recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 4503–4511, 2021.

[42] Junliang Yu, Hongzhi Yin, Jundong Li, Qinyong Wang, Nguyen Quoc Viet Hung, and Xiangliang Zhang. Self-supervised multi-channel hypergraph convolutional network for social recommendation. In *Proceedings of the Web Conference*, pages 413–424, 2021.

[43] Jing Wang, Xin Geng, and Hui Xue. Re-weighting large margin label distribution learning for classification. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021. doi: 10.1109/TPAMI.2021.3082623.

[44] Li Yuan, Francis EH Tay, Guilin Li, Tao Wang, and Jiashi Feng. Revisiting knowledge distillation via label smoothing regularization. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3903–3911, 2020.

[45] Rafael Müller, Simon Kornblith, and Geoffrey E Hinton. When does label smoothing help? *Advances in neural information processing systems*, 32, 2019.

[46] Youngduck Choi, Youngnam Lee, Dongmin Shin, Junghyun Cho, Seoyon Park, Seewoo Lee, Jineon Baek, Chan Bae, Byungsoo Kim, and Jaewe Heo. Ednet: A large-scale hierarchical dataset in education. In *Proceedings of the 21st International Conference on Artificial Intelligence in Education*, pages 69–73, 2020.

[47] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[48] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, pages 3538–3545, 2018.

[49] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(11):2579–2605, 2008.

[50] Aritra Ghosh, Jay Raspat, and Andrew Lan. Option tracing: Beyond correctness analysis in knowledge tracing. In *International Conference on Artificial Intelligence in Education*, pages 137–149, 2021.

[51] Zichao Wang, Angus Lamb, Evgeny Saveliev, Pashmina Cameron, Jordan Zaykov, Jose Miguel Hernandez-Lobato, Richard E. Turner, Richard G. Baraniuk, Eedi Craig Barton, Simon Peyton Jones, Simon Woodhead, and Cheng Zhang. Results and insights from diagnostic questions: The neurips 2020 education challenge. In *NeurIPS 2020 Competition and Demonstration Track*, volume 133, pages 191–205, 2021.

[52] Ting Long, Jiarui Qin, Jian Shen, Weinan Zhang, Wei Xia, Ruiming Tang, Xiuqiang He, and Yong Yu. Improving knowledge tracing with collaborative information. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*, page 599–607, 2022.